Symbiosis Robotics
FROM HUMANOID TO HUMAN

# ODEAction: From Predicting Chunks to Evolving Actions

**Qishen Wang**[1,2], **Pengxiang Ding**[1,3,4‡], **Shifeng Bao**[5], **Mingyang Sun**[1,3,4], **Hengtao Li**[1], **Yi Gao**[1], **Baohua Yin**[1], **Yang Liu**[4], **Jiajun Fu**[1,6], **Jiacheng Liu**[1,3,4], **Changjing He**[1,6], **Peiyang Li**[1,7], **Runze Suo**[8], **Yan Wang**[6], **Xiaobo Wang**[9], **Yonghao Dang**[10], **Cheng Chi**[5*]

[1] Symbiosis Robotics, [2] University of Science and Technology of China, [3] Zhejiang University, [4] Westlake University, [5] Renmin University of China, [6] Tsinghua University, [7] Beijing University of Posts and Telecommunications, [8] Fudan University, [9] Shenzhen University of Advanced Technology, [10] Central University of Finance and Economics

‡ Project Lead, * Corresponding Author

## Abstract

Vision-language-action (VLA) models typically generate actions in chunks, organizing continuous robot execution into separately predicted segments and thus introducing potential discontinuities at action-chunk transitions. We propose ODEAction, a chunk-free action generation paradigm that replaces action-chunk prediction with Continuous Action Evolution. Given visual observations, language instructions, and robot proprioceptive information, ODEAction learns an action transition field defined over action space, where a transition vector characterizes the evolution rate of the current action with respect to evolution time. During execution, the model takes the current action as its initial condition and integrates the learned field using an ordinary differential equation (ODE), recursively generating subsequent actions with explicit dependencies. ODEAction is implemented as an action head and does not depend on a specific vision-language backbone architecture. Experiments on LIBERO, EBench, RoboTwin, and real robots show that ODEAction improves both task performance and trajectory quality over matched action-chunking baselines, with particularly pronounced gains on fine manipulation tasks.

## 1 Introduction

Vision-language-action (VLA) models [22, 24–26, 38, 41, 53] widely adopt action-chunk generation (Action Chunking Generation): they jointly predict a sequence of future actions in a single inference pass, organizing continuous execution into successively generated action chunks [6–8, 23, 39, 40, 46]. However, adjacent chunks predicted independently typically lack explicit temporal consistency constraints across chunk boundaries, potentially causing abrupt action changes and trajectory discontinuities at transitions [9, 31] (Figure 1(a)).

Existing methods alleviate this issue by imposing constraints on continuity across chunks: RTC [9] constrains new action chunks during inference, whereas Training-time RTC [10] and Legato [32] introduce continuity constraints during training. Nevertheless, these methods still treat action chunks as the fundamental unit of generation, improving the connections between chunks rather than changing the action generation paradigm. This leads us to ask a further question: can we move beyond chunk-wise prediction and directly model the continuous evolution of actions over time?

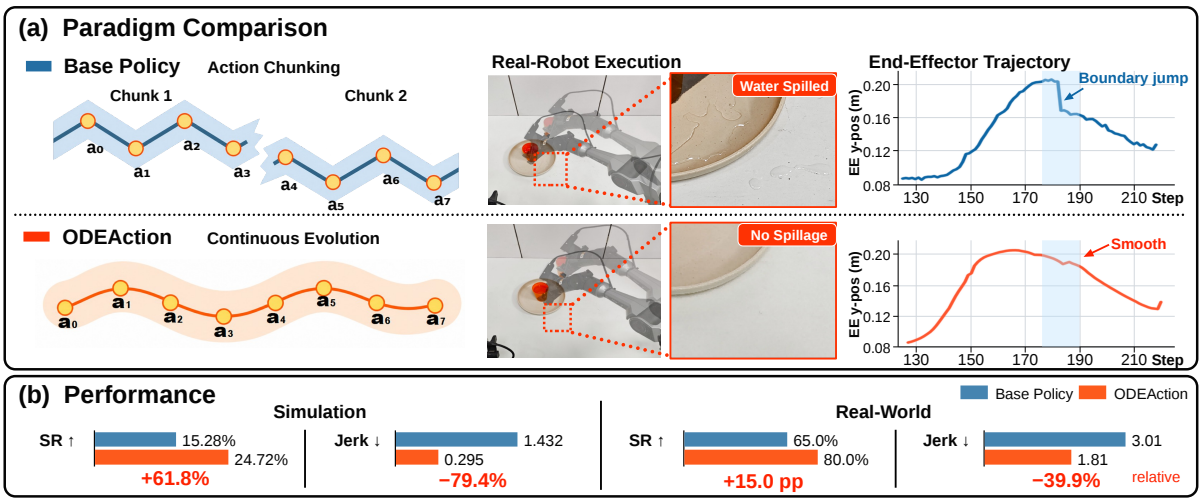



**Figure 1.** From action-chunk prediction to continuous action evolution. (a) Base Policy uses action-chunk prediction and is prone to abrupt action changes at chunk transitions, whereas ODEAction's continuous evolution ensures that action trajectories are generated continuously. Comparisons of real-robot task execution and the corresponding end-effector trajectories illustrate the differences in execution smoothness and trajectory continuity between the two methods. (b) Representative results on EBench simulation and real-robot tasks: compared with Base Policy, ODEAction increases the task success rate (SR) and reduces the end-effector mean absolute jerk (EE-MAJ).

Robot execution itself progresses incrementally over time rather than occurring in action chunks. We therefore characterize continuous action evolution using an ordinary differential equation (ODE): the current action provides the starting point, a learned action evolution rate determines subsequent changes, and integration progressively generates an action sequence without jointly predicting an entire future action chunk. Action generation thus shifts from predicting future segments to learning the dynamics that drive actions to evolve continuously.

From this perspective, we propose **ODEAction**, an action generation paradigm centered on **Continuous Action Evolution**. Given visual observations, language instructions, and proprioceptive information, ODEAction learns an Action Transition Field defined over action space. At each action and evolution time, the field outputs a transition vector representing the evolution rate of the current action with respect to evolution time. During training, we construct a target field around demonstration trajectories that combines progression along the demonstrations with deviation correction, and learn this field through direct supervision. During inference, starting from the current action, we integrate the learned action transition field using an ODE, so that each generated action becomes the starting point for subsequent evolution, instead of independently predicting and concatenating action chunks (Figure 1(a)). ODEAction is implemented as an action head that can be integrated with different vision-language backbones.

We evaluate ODEAction on simulation benchmarks, including LIBERO [29], RoboTwin [12], and EBench [16], as well as on real-robot tasks. Compared with matched action-chunking baselines, ODEAction improves both task performance and action trajectory quality, achieving a 61.8% relative improvement in average success rate on EBench. Further analysis shows that evaluation units exhibiting larger improvements in boundary continuity also tend to achieve greater success-rate gains. In a representative execution of the real-world Place Cup task, ODEAction smoothly transports a water-filled cup without spilling, while producing a more continuous end-effector trajectory (Figure 1(a)). Our contributions are threefold:

- We propose ODEAction, a chunk-free generation paradigm based on continuous action evolution, transforming action generation from future action-chunk prediction into a learnable continuous evolution process.

- Extensive experiments on simulation benchmarks and real-robot tasks show that ODEAction improves both task performance and trajectory quality over matched action-chunking baselines, with particularly substantial benefits on fine manipulation tasks.
- We identify a systematic relationship between improvements in trajectory quality and gains in task performance: boundary discontinuity reduction is strongly positively correlated with success-rate gains, and evaluation units with greater improvements in trajectory continuity generally achieve larger task-performance gains.

## 2 Method

ODEAction models action generation as continuous evolution starting from the current action. Figure 2 illustrates its pipeline for constructing the demonstration-induced target field, learning through direct supervision, and recursively generating actions via ODE integration.

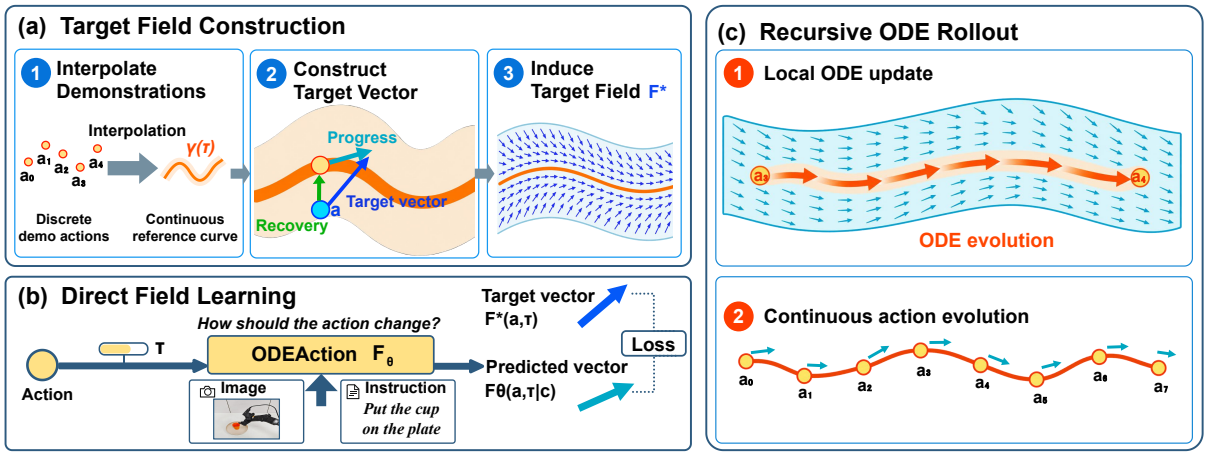



**Figure 2.** Learning the action transition field and recursively generating actions. (a) Discrete demonstrated actions are interpolated into a continuous trajectory. Target transition vectors are constructed by combining progression along the demonstration with deviation correction, inducing a target action transition field $F^*$ in the trajectory neighborhood. (b) During training, the action transition field ($F_\theta$) is learned through direct supervision. (c) During inference, the current action serves as the initial condition, and subsequent actions are recursively generated by numerically integrating the learned action transition field using an ODE.

### 2.1 Continuous Action Evolution

Unlike methods that jointly predict a future action chunk [7, 13, 23, 35, 51], ODEAction uses an ordinary differential equation to describe the continuous evolution of the current action over execution time. Let $c$ denote the conditioning information provided by visual observations, language instructions, and robot proprioceptive information, and let normalized evolution time $\tau \in [0, 1]$ parameterize this process. Action evolution then satisfies

$$\frac{\mathrm{d}a(\tau)}{\mathrm{d}\tau} = F_\theta\big(a(\tau), \tau \mid c\big), \tag{1}$$

where $a(\tau)$ denotes the action at evolution time $\tau$, and $F_\theta$ is a conditional vector field defined over action space, termed the **Action Transition Field**. Given an action and an evolution time, this field outputs a transition vector with the same dimensionality as the action, representing the action evolution rate of the current action with respect to evolution time.

## 2.2 Demonstration-Induced Action Transition Field

Training demonstrations consist of discrete action sequences. We interpolate between adjacent demonstrated actions along evolution time to obtain a continuous demonstration trajectory $\gamma(\tau)$. Its derivative $\dot{\gamma}(\tau) = \mathrm{d}\gamma(\tau)/\mathrm{d}\tau$ gives the action evolution rate of the demonstration at evolution time $\tau$, providing supervision for the action transition field in Equation (1) along the trajectory.

However, prediction errors or execution deviations during inference may cause the current action to drift away from the demonstration trajectory during recursive evolution. Supervising only the evolution rate on the trajectory cannot provide a corresponding evolution direction at off-trajectory positions. We therefore sample query actions $a = \gamma(\tau) + \sigma\epsilon$ around $\gamma(\tau)$, where $\epsilon \sim \mathcal{N}(0, I)$ and $\sigma$ controls the extent of neighborhood coverage. To enable off-trajectory actions both to progress along the demonstration and to return toward its path, we define the target action transition field as

$$F^*(a, \tau) = \dot{\gamma}(\tau) + k\big(\gamma(\tau) - a\big), \tag{2}$$

where the first term specifies the direction of progression along the demonstration, and the second drives deviating actions back toward the corresponding demonstration path; $k$ controls the correction strength [19]. When $a = \gamma(\tau)$, the correction term vanishes, and the target transition vector equals the demonstration's evolution rate. When the action deviates, the two terms jointly determine the direction of subsequent evolution. In this way, discrete demonstrations induce a target action transition field in the neighborhoods of their trajectories (Figure 2(a)). Supervision samples from different demonstrations, conditions, and evolution times are jointly used to learn the conditional action transition field $F_\theta$. ODEAction uses a fixed neighborhood scale $\sigma$ and correction gain $k$ throughout the entire evolution interval.

## 2.3 Direct Supervision and ODE Rollout

**Direct Supervision.** Given conditioning information $c$ and a demonstration trajectory $\gamma$ from the training data, we sample $\tau \sim \mathcal{U}[0, 1]$, then sample a query action $a \sim \mathcal{N}(\gamma(\tau), \sigma^2 I)$ in the neighborhood of the corresponding demonstrated action. We directly supervise $F_\theta$ using the target field $F^*$:

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{(c,\gamma)\sim\mathcal{D},\\ \tau\sim\mathcal{U}[0,1],\\ a\sim\mathcal{N}(\gamma(\tau),\sigma^2 I)}} \left[ \left\| F_\theta(a, \tau \mid c) - F^*(a, \tau) \right\|_2^2 \right]. \tag{3}$$

Each training sample provides a query action and its target transition vector, from which the model learns evolution rates under different conditions, action positions, and evolution times (Figure 2(b)). Training requires neither solving an ODE nor backpropagating through numerical integration.

**ODE Rollout.** During inference, we initialize the process with the current action and integrate the learned action transition field over evolution time to generate subsequent actions step by step. Taking the forward Euler method as an example, one integration step is

$$a_{n+1} = a_n + \Delta\tau \, F_\theta(a_n, \tau_n \mid c). \tag{4}$$

Equation (4) is the forward Euler discretization of Equation (1). The same action transition field can also be integrated using higher-order methods, such as Runge–Kutta, without changing the learned action evolution model. Each generated action serves as the starting point for subsequent evolution, establishing explicit generative dependencies between adjacent actions (Figure 2(c)). When a new visual observation becomes available, the model updates the condition $c$ using the latest observation and continues integrating from the current action along the updated conditional action transition field. Each query to the action transition field outputs only one $d$-dimensional transition vector; subsequent

actions are produced through recursive integration rather than through joint prediction of a fixed-length action chunk.

## 3 Experiments

This section addresses the following questions:

- Can ODEAction improve task performance and trajectory quality simultaneously? Are improvements in trajectory continuity associated with success-rate gains? (Section 3.2)
- Does ODEAction remain effective across different base policies and tasks? (Section 3.3)
- Can ODEAction outperform methods designed to maintain continuity across action chunks? (Section 3.4)
- Which design choices are necessary for ODEAction's performance? (Section 3.5)
- Can ODEAction improve both task performance and trajectory quality on real-robot tasks? (Section 3.6)

### 3.1 Experimental Setup

**Policies and Protocols.** We consider three base-policy configurations. With $\pi_{0.5}$ [8] as the base policy, we compare action-chunk generation and ODEAction on LIBERO and EBench, and additionally include RTC and Legato in the real-robot experiments. With StarVLA-OFT [14, 23] as the base policy, we evaluate ODEAction on all 50 RoboTwin tasks. With StarVLA-$\pi$ as the base policy, we compare the original action-chunking method, RTC, Legato, and ODEAction on five representative RoboTwin tasks. LIBERO [29], RoboTwin [12], and EBench [16] cover tabletop manipulation, bimanual manipulation, and long-horizon fine manipulation, respectively. Real-robot experiments use AgileX Cobot Magic 1.0, and each benchmark follows its corresponding evaluation protocol. Within each comparison, we keep the base backbone, task-training data, control frequency, and visual-observation update times identical; details of evolution time and the execution mechanism are provided in Supplementary Materials A and B. Except for ablation experiments, ODEAction consistently uses $\sigma = 0.1$ and $k = 5$ across all benchmarks, tasks, and real-robot experiments, without task- or platform-specific adjustments; parameter selection is discussed in Supplementary Material A.

**Metrics.** We measure task performance using the success rate (SR) across all evaluation episodes. For trajectory quality, we use four metrics, all of which are better when lower. NLDLJ, NSPARC [2, 5], and EE-MAJ measure time-domain smoothness, frequency-domain smoothness, and end-effector mean absolute jerk, respectively, and are collectively referred to as Motion Smoothness metrics. Boundary Jump measures the magnitude of changes between adjacent control commands at chunk boundaries, characterizing Trajectory Continuity. For ODEAction, Boundary Jump is computed at the chunk-boundary times of the corresponding Base Policy. Trajectory metrics are calculated from model-output control commands in the successful episodes of each method; the separate evaluation protocol for offline recursive error is described in Supplementary Material C.

### 3.2 Comparison with Action Chunking

Using $\pi_{0.5}$ as the base policy, we compare task performance and trajectory quality between action-chunk generation and ODEAction on LIBERO and EBench, and analyze the relationship between improvements in boundary continuity and success-rate gains. To quantify improvements in boundary continuity relative to the action-chunking baseline, we define the Boundary Discontinuity Reduction (BDR) for evaluation unit $T$ as

$$\mathrm{BDR}(T) = \frac{J^{\mathrm{bnd}}_{\mathrm{chunk}}(T) - J^{\mathrm{bnd}}_{\mathrm{ODEAction}}(T)}{J^{\mathrm{bnd}}_{\mathrm{chunk}}(T)}. \tag{5}$$

Here, $J^{\mathrm{bnd}}_{\mathrm{chunk}}$ and $J^{\mathrm{bnd}}_{\mathrm{ODEAction}}$ denote the Boundary Jump of Base Policy and ODEAction, respectively, both evaluated at the same Base Policy action-chunk boundary times. A larger BDR indicates a greater reduction in boundary discontinuity.

**Task Performance and Trajectory Quality.** As shown in Table 1, the average success rate on LIBERO increases from 96.85% to 98.25%, with improvements across all four suites. On EBench, it increases from 15.28% to 24.72%, representing a 61.8% relative improvement. Across all evaluation units in both benchmarks, ODEAction simultaneously increases the success rate and improves all four trajectory metrics, with particularly pronounced performance gains on fine manipulation tasks.

**Table 1.** Comparison of task success rates and trajectory quality between ODEAction and Base Policy on LIBERO and EBench. SR gains are expressed in percentage points (pp), while improvements in NLDLJ, NSPARC, and EE-MAJ are relative reductions; EE-MAJ is reported in units of $10^{-3}$. The mean BDR is the arithmetic mean across evaluation units, and aggregate results are calculated from unrounded data.

| | SR (%) ↑ | | NLDLJ ↓ | | NSPARC ↓ | | EE-MAJ ↓ | | Boundary Jump ↓ | | BDR (%) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Benchmark / Task | Base | Ours | Base | Ours | Base | Ours | Base | Ours | Base | Ours | |
| *LIBERO* | | | | | | | | | | | |
| Spatial | 98.8 | **99.3** | 26.35 | **25.32** | 2.121 | **1.983** | 1.19 | **0.48** | 0.04314 | **0.04283** | 0.72 |
| Object | 98.2 | **99.8** | 25.87 | **24.46** | 2.104 | **2.010** | 1.14 | **0.39** | 0.04102 | **0.04040** | 1.51 |
| Goal | 98.0 | **98.8** | 26.72 | **25.86** | 2.138 | **1.962** | 1.23 | **0.53** | 0.04577 | **0.04547** | 0.66 |
| Long | 92.4 | **95.1** | 28.24 | **26.44** | 2.172 | **2.042** | 1.31 | **0.38** | 0.05213 | **0.05061** | 2.92 |
| Average | 96.85 | **98.25** | 26.80 | **25.52** | 2.134 | **1.999** | 1.22 | **0.44** | 0.04552 | **0.04483** | 1.45 |
| *Relative improvement* | *+1.40 pp* | | *4.76%* | | *6.31%* | | *63.70%* | | — | | — |
| *EBench* | | | | | | | | | | | |
| Coffee | 5.0 | **13.3** | 27.88 | **26.24** | 2.138 | **2.025** | 1.38 | **0.29** | 0.01680 | **0.01390** | 17.26 |
| Flip Cup | 5.0 | **10.0** | 29.15 | **26.73** | 2.118 | **2.034** | 1.62 | **0.34** | 0.02000 | **0.01810** | 9.50 |
| Frame | 30.0 | **48.3** | 26.70 | **25.65** | 2.122 | **2.022** | 1.53 | **0.31** | 0.01520 | **0.01140** | 25.00 |
| Gear | 15.0 | **23.3** | 24.36 | **22.91** | 2.125 | **2.014** | 1.50 | **0.29** | 0.01400 | **0.01300** | 7.14 |
| Glass | 25.0 | **30.0** | 28.55 | **26.07** | 2.121 | **2.027** | 1.11 | **0.26** | 0.02450 | **0.02290** | 6.53 |
| Nut | 11.7 | **23.3** | 26.29 | **25.47** | 2.145 | **2.016** | 1.46 | **0.28** | 0.01180 | **0.00990** | 16.10 |
| Average | 15.28 | **24.72** | 27.16 | **25.51** | 2.128 | **2.023** | 1.43 | **0.30** | 0.01705 | **0.01487** | 13.59 |
| *Relative improvement* | *+9.44 pp* | | *6.08%* | | *4.93%* | | *79.40%* | | — | | — |

**Association Between BDR and Task-Performance Gains.** The average BDR values on LIBERO and EBench are 1.45% and 13.59%, respectively, corresponding to average success-rate gains of +1.40 and +9.44 percentage points. As shown in Figure 3(a), when ranked in descending order of BDR, all six EBench tasks precede the four LIBERO suites; Figure 3(b) shows the corresponding success-rate gains for each evaluation unit in the same order. Across all ten evaluation units, BDR is strongly positively correlated with success-rate gains (Spearman $\rho_s = 0.939$; detailed statistics are provided in Supplementary Material D). This indicates that evaluation units with greater improvements in boundary continuity also tend to achieve larger success-rate gains.

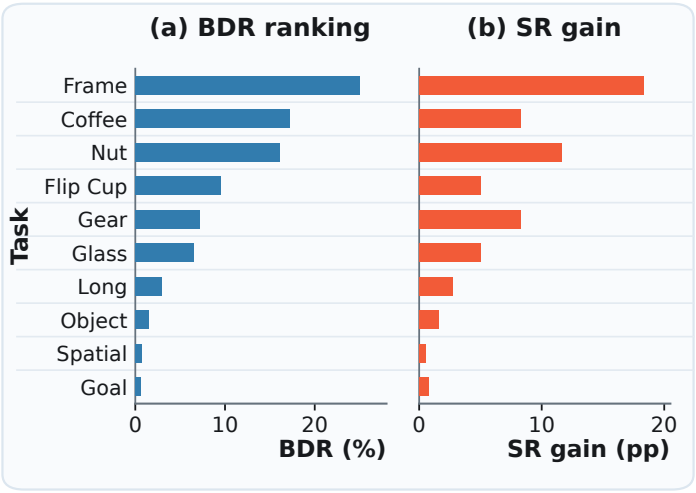



**Figure 3.** Relationship between improvements in boundary continuity and gains in task performance. (a) Four LIBERO suites and six EBench tasks arranged in descending order of Boundary Discontinuity Reduction (BDR). (b) Success-rate gains (SR gain) of ODEAction relative to Base Policy, shown in the same evaluation-unit order. BDR and success-rate gains exhibit a strong positive correlation across the ten evaluation units.

### 3.3 Generality of the Generation Paradigm

To examine generality across policies and tasks, we use StarVLA-OFT as the Base Policy and evaluate ODEAction on all 50 RoboTwin tasks. As shown in Table 2, ODEAction increases the average success rate across the 50 RoboTwin tasks from 85.7%/87.3% to 89.4%/89.8% under the Clean/Random settings, respectively (per-task results are provided in Supplementary Material E). All four control-command trajectory metrics also improve: NLDLJ decreases from 25.96 to 23.89, NSPARC from 2.298 to 2.159, EE-MAJ from 1.145 to 0.635, and Boundary Jump from 0.11200 to 0.08710. Together with the findings in Section 3.2, these results show that ODEAction's benefits for task performance and trajectory quality extend across base policies and benchmarks.

**Table 2.** Success rates (%) on all 50 RoboTwin tasks. Base Policy denotes StarVLA-OFT [14].

| Method | Clean ↑ | Random ↑ |
|---|---|---|
| $\pi_0$ [7] | 65.9 | 58.4 |
| $\pi_{0.5}$ [8] | 82.7 | 76.8 |
| ABot-M0 [48] | 86.1 | 85.1 |
| X-VLA [52] | 72.9 | 72.8 |
| LingBot-VLA [47] | 88.6 | 86.7 |
| Base Policy [49] | 85.7 | 87.3 |
| ODEAction (Ours) | **89.4** | **89.8** |

### 3.4 Comparison with Action-Chunk Continuation Methods

**Comparison Setup.** To compare action-chunk continuation with continuous action evolution, we use StarVLA-$\pi$ as the Base Policy and establish four comparison groups on five RoboTwin tasks featuring different motion constraints and bimanual coordination patterns: Base Policy uses the original action-chunking method; Base Policy + RTC [9] constrains new action chunks using already determined actions during inference; Base Policy + Legato [32] incorporates known-action conditions during training to learn cross-chunk continuation; and Base Policy + ODEAction replaces the action

head with ODEAction. All four groups share the same vision-language backbone, training data, control frequency, and observation-update times. Task and implementation details are provided in Supplementary Material B.

**Table 3.** Comparison of four methods on five RoboTwin tasks, using StarVLA-$\pi$ as the Base Policy. EE-MAJ is reported in units of $10^{-3}$.

| | SR (%) ↑ | | | | NLDLJ ↓ | | | | NSPARC ↓ | | | | EE-MAJ ↓ | | | | Boundary Jump ↓ | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Task | Base | +RTC | +Legato | Ours | Base | +RTC | +Legato | Ours | Base | +RTC | +Legato | Ours | Base | +RTC | +Legato | Ours | Base | +RTC | +Legato | Ours |
| Adjust Bottle | 92 | 91 | 93 | **94** | 23.68 | 23.66 | 23.71 | **21.54** | 1.998 | 2.030 | 2.051 | **1.991** | 1.095 | 1.068 | 1.020 | **0.625** | 0.1720 | 0.1562 | 0.1616 | **0.1507** |
| Handover Block | 79 | 80 | 75 | **82** | 29.10 | 28.17 | 27.59 | **26.67** | 2.850 | 2.954 | 2.868 | **2.735** | 1.248 | 1.050 | 0.833 | **0.581** | 0.0677 | 0.0630 | 0.0612 | **0.0391** |
| Open Laptop | 85 | 82 | 78 | **87** | 25.53 | 25.15 | 25.30 | **22.73** | 2.023 | 2.009 | 1.996 | **1.848** | 1.037 | 0.987 | 1.046 | **0.449** | 0.0781 | 0.0681 | 0.0660 | **0.0264** |
| Lift Pot | 88 | 90 | 86 | **91** | 23.34 | 22.64 | 22.59 | **20.69** | 1.723 | 1.681 | 1.698 | **1.566** | 1.170 | 0.817 | 0.724 | **0.511** | 0.1904 | 0.1712 | 0.1725 | **0.1613** |
| Place Can Basket | **68** | 63 | 58 | **68** | 25.19 | 24.63 | 24.57 | **23.18** | 2.554 | 2.502 | 2.550 | **2.338** | 0.848 | 0.709 | 0.662 | **0.500** | 0.0742 | 0.0540 | 0.0497 | **0.0462** |
| Average | 82.4 | 81.2 | 78.0 | **84.4** | 25.37 | 24.85 | 24.75 | **22.96** | 2.230 | 2.235 | 2.233 | **2.096** | 1.080 | 0.926 | 0.857 | **0.533** | 0.1165 | 0.1025 | 0.1022 | **0.0847** |

**Results.** As shown in Table 3, the average success rates of Base Policy, +RTC, +Legato, and +ODEAction are 82.4%, 81.2%, 78.0%, and 84.4%, respectively. Although RTC and Legato reduce Boundary Jump from 0.1165 to 0.1025 and 0.1022, their success rates decrease by 1.20 and 4.40 percentage points, respectively. In contrast, ODEAction increases the success rate to 84.4% while reducing Boundary Jump to 0.0847 (a 27.3% reduction). Across the five tasks, ODEAction achieves the sole highest success rate on four tasks and ties for the highest on one; it also achieves the best values for all four trajectory-quality metrics on every task. These results demonstrate simultaneous improvements in task performance, motion smoothness, and trajectory continuity through continuous action evolution.

### 3.5 Ablation Studies

**Ablation Setup.** Two key design choices jointly shape ODEAction's target action transition field: the action neighborhood scale $\sigma$ determines the extent of supervision coverage around demonstration trajectories, while the correction gain $k$ determines how strongly deviating actions are driven back toward the demonstration evolution path. To validate the roles of these two choices, we conduct ablations using the base policy and five tasks from Section 3.4. We compare the full ODEAction with two variants: w/o neighborhood sets $\sigma = 0$, providing supervision only on the demonstration trajectory; w/o correction sets $k = 0$, retaining neighborhood sampling but removing the correction term. Because the correction term naturally vanishes when $\sigma = 0$, these three groups cover all effective ablation combinations of the two designs. We additionally include Direct Next-Action, which likewise removes action chunks, to examine whether ODEAction's performance advantage stems solely from action-by-action recursion.

**Table 4.** Ablation of key ODEAction design choices. The w/o neighborhood variant sets $\sigma = 0$, and the w/o correction variant sets $k = 0$. $E$ denotes the full-trajectory offline rollout error, while $E_1$ and $E_2$ denote errors over the first and second halves, respectively; all are reported in units of $10^{-4}$. $E_2/E_1$ is the error accumulation ratio.

| Method | SR (%) ↑ | $E \downarrow$ | $E_1 \downarrow$ | $E_2 \downarrow$ | $E_2/E_1 \downarrow$ |
|---|---|---|---|---|---|
| w/o neighborhood | 47.4 | 8.84 | 5.78 | 11.90 | 2.06 |
| w/o correction | 56.2 | 13.17 | 8.26 | 18.08 | 2.19 |
| ODEAction | **84.4** | **4.25** | **4.42** | **4.07** | **0.92** |

**Metrics.** In addition to success rate, we perform an offline rollout starting from the beginning of a demonstration trajectory under fixed conditions and compare the recursively generated actions with the demonstrated actions at corresponding evolution times, obtaining error $E$. Denoting the errors over the first and second halves of the rollout by $E_1$ and $E_2$, we define the Error Accumulation Ratio as

$E_2/E_1$. A smaller ratio indicates weaker relative error growth in the second half, while a ratio greater than 1 indicates a larger error in the second half. The detailed calculation is provided in Supplementary Material C.

**Both the Action Neighborhood and Correction Term Are Necessary.** As shown in Table 4, removing the neighborhood or correction term reduces the success rate from 84.4% to 47.4% or 56.2%, respectively, while increasing offline error from 4.25 to 8.84 or 13.17. The error accumulation ratios of the two ablated variants are 2.06 and 2.19, compared with only 0.92 for the full ODEAction. Neighborhood supervision covers off-trajectory actions, while the correction term provides a direction for returning to the demonstration path. Together, they suppress the sustained growth of recursive errors.

**Distinction from Ordinary Action-by-Action Recursion.** Direct Next-Action recursively generates actions by directly predicting the next action. Under the same five-task setting, it achieves an average success rate of 78.4%, below ODEAction's 84.4%, and performs worse on all four trajectory-quality metrics (Supplementary Material F). This result indicates that ODEAction's benefits do not arise solely from removing action chunks, but from the combined effects of modeling continuous action evolution rates and supervising the demonstration-neighborhood target field.

### 3.6 Real-Robot Experiments

**Setup.** On the AgileX Cobot Magic 1.0 platform (Figure 4(a)), we use $\pi_{0.5}$ as the Base Policy and compare the original action-chunking method, RTC, Legato, and ODEAction. The four real-robot tasks (Figure 4(b)) are Put Bananas, Open Drawer, Place Cup, and Insert Spoon, covering pick-and-place, articulated-object manipulation, placement of a water-filled cup, and precise insertion. Place Cup requires that no water be spilled throughout execution. We collect 100 demonstration trajectories for each task and test each method 20 times per task, calculating success rates from the number of successful trials.

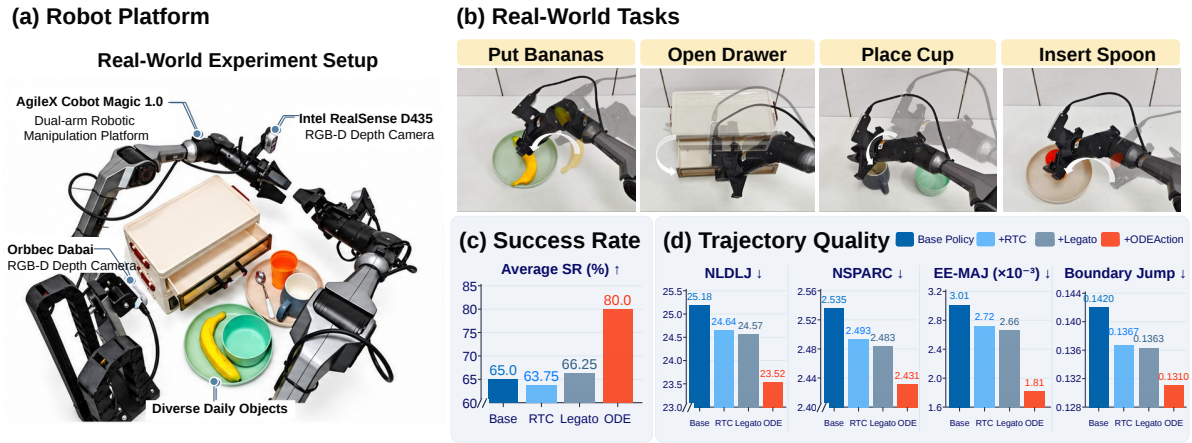



**Figure 4.** Real-robot experimental platform, tasks, and quantitative evaluation. (a) AgileX Cobot Magic 1.0 dual-arm robotic platform and visual sensor configuration. (b) Real-robot tasks. (c) Comparison of average success rates (SR) across the four tasks. (d) Comparison of average trajectory quality across the four tasks.

**Results.** As shown in Figure 4(c), Base Policy, +RTC, +Legato, and +ODEAction achieve average success rates of 65.00%, 63.75%, 66.25%, and 80.00%, respectively. ODEAction improves upon Base Policy by 15.00 percentage points and also outperforms RTC and Legato. In terms of trajectory quality, ODEAction outperforms the other methods on all four control-command trajectory metrics in each of the four tasks (complete per-task results are provided in Supplementary Table S2). The averaged results are shown in Figure 4(d): EE-MAJ decreases from 3.01 to 1.81, and Boundary Jump decreases from 0.14200 to 0.13100. Gains are particularly pronounced in fine manipulation: the success rate on Place Cup rises

from 70.0% to 90.0% (see Figure 1(a) for execution and trajectory comparisons), and that on Insert Spoon rises from 25.0% to 55.0%.

## 4 Related Work

### 4.1 Action Chunking and Cross-Chunk Continuation

Action chunking [13, 51] has become a common approach to action generation in large robot foundation models [7, 11, 23, 24, 30, 37, 42, 43, 45, 50], with models such as GR00T N1 [6] and $\pi_{0.5}$ [8] jointly generating a sequence of future actions in a single pass. However, independently predicting adjacent action chunks can lead to action discontinuities at chunk transitions. To improve continuity across chunks, RTC [9] constrains new chunk generation using the overlapping portion of the previous action chunk during inference, while other methods enhance cross-chunk consistency through candidate selection [31] or generation guidance [2, 34]. Training-time RTC [10] and Legato [32] incorporate known actions as conditioning information during training, enabling policies to learn chunk continuation; related studies also combine masked action learning with prefix-preserved sampling to improve action consistency during asynchronous execution [44]. These methods retain action chunks as the unit of generation, whereas ODEAction recursively generates subsequent actions through continuous action evolution.

### 4.2 Dynamical Systems and Continuous Action Generation

Continuous dynamical systems have a long history in modeling robot motion. Prior work uses movement primitives and stable motion fields to reproduce demonstrated behaviors [17, 21], and explores constructing motion fields that combine progression along reference trajectories with recovery from deviations [20, 27, 28]. This perspective has also been incorporated into visuomotor policies: NDP [3] and related approaches embed movement primitives into neural policies [4], while SFP [19], SSIP [18], and other methods enable streaming action generation through continuous dynamics in action space [33]. Under vision-language conditioning, studies have further explored the continuous evolution of proprioceptive representations [36] and the integration of vision-language reasoning with movement primitive generation [1, 15]. ODEAction models the action generation process using an ordinary differential equation, recursively generating subsequent actions from the current action to improve task success rates and trajectory quality.

## 5 Conclusion

We propose ODEAction, a chunk-free action generation paradigm that replaces future action-chunk prediction with Continuous Action Evolution. Conditioned on visual, language, and proprioceptive information, ODEAction learns an action transition field and recursively generates subsequent actions through ODE integration, using the current action as the initial condition. Experiments on LIBERO, EBench, RoboTwin, and real robots show that ODEAction improves both task performance and trajectory quality over action-chunking baselines, with particularly pronounced benefits on fine manipulation tasks. We further find a strong positive correlation between boundary discontinuity reduction and success-rate gains: evaluation units with larger improvements in trajectory continuity also tend to achieve greater success-rate gains. Overall, ODEAction offers a new modeling paradigm for action generation in VLAs and robot control.

## References


[1] Edgar Anarossi, Yuhwan Kwon, Hirotaka Tahara, Shohei Tanaka, Keisuke Shirai, Masashi Hamaya, Cristian C Beltran-Hernandez, Atsushi Hashimoto, and Takamitsu Matsubara. Keymps: One-shot vision-language guided motion generation by sequencing dmps for occlusion-rich tasks. *IEEE Access*, 2025.

[2] Nadun Ranawaka Arachchige, Zhenyang Chen, Wonsuhk Jung, Woo Chul Shin, Rohan Bansal, Pierre Barroso, Yu Hang He, Yingyang Celine Lin, Benjamin Joffe, Shreyas Kousik, et al. Sail: Faster-than-demonstration execution of imitation learning policies. *arXiv preprint arXiv:2506.11948*, 2025.

[3] Shikhar Bahl, Mustafa Mukadam, Abhinav Gupta, and Deepak Pathak. Neural dynamic policies for end-to-end sensorimotor learning. *Advances in Neural Information Processing Systems*, 33:5058–5069, 2020.

[4] Shikhar Bahl, Abhinav Gupta, and Deepak Pathak. Hierarchical neural dynamic policies. *arXiv preprint arXiv:2107.05627*, 2021.

[5] Sivakumar Balasubramanian, Alejandro Melendez-Calderon, Agnès Roby-Brami, and Etienne Burdet. On the analysis of movement smoothness. *Journal of NeuroEngineering and Rehabilitation*, 12, 2015.

[6] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[7] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. $\pi_0$: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[8] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*, 2025.

[9] Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.

[10] Kevin Black, Allen Z Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.

[11] Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liqun Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, et al. Gr-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.

[12] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[13] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[14] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

[15] Yinlong Dai, Benjamin A Christie, Daniel J Evans, Dylan P Losey, and Simon Stepputtis. Language movement primitives: Grounding language models in robot motion. *arXiv preprint arXiv:2602.02839*, 2026.

[16] Ning Gao, Jinliang Zheng, Xing Gao, Haoxiang Ma, Hanqing Wang, Yukai Wang, Jiantong Chen, Zanxin Chen, Shujie Zhang, Mingda Jia, et al. Ebench: Elemental diagnosis of generalist mobile manipulation policies. *arXiv preprint arXiv:2606.18239*, 2026.

[17] Auke Jan Ijspeert, Jun Nakanishi, Heiko Hoffmann, Peter Pastor, and Stefan Schaal. Dynamical movement primitives: learning attractor models for motor behaviors. *Neural computation*, 25(2):328–373, 2013.

[18] Puming Jiang, Meiyi Wang, Kelvin Lin, Ce Hao, and Harold Soh. Guided streaming stochastic interpolant policy. *arXiv preprint arXiv:2605.10051*, 2026.

[19] Sunshine Jiang, Xiaolin Fang, Nicholas Roy, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Siddharth Ancha. Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories. *arXiv preprint arXiv:2505.21851*, 2025.

[20] Yuri A Kapitanyuk, Anton V Proskurnikov, and Ming Cao. A guiding vector-field algorithm for path-following control of nonholonomic mobile robots. *IEEE Transactions on Control Systems Technology*, 26(4):1372–1385, 2017.

[21] S Mohammad Khansari-Zadeh and Aude Billard. Learning stable nonlinear dynamical systems with gaussian mixture models. *IEEE Transactions on Robotics*, 27(5):943–957, 2011.

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[24] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation, 2024.

[25] Xinghang Li, Peiyan Li, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, Tao Kong, Hanbo Zhang, and Huaping Liu. Towards generalist robot policies: What matters in building vision-language-action models. *arXiv preprint arXiv:2412.14058*, 3(7):8, 2024.

[26] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. In *The Twelfth International Conference on Learning Representations*, 2024.

[27] Yiming Li and Sylvain Calinon. From movement primitives to distance fields to dynamical systems. *IEEE Robotics and Automation Letters*, 2025.

[28] Yiming Li, Nael Darwiche, Amirreza Razmjoo, Sichao Liu, Yilun Du, Auke Ijspeert, and Sylvain Calinon. Geometry-aware policy imitation. In *International Conference on Learning Representations*, volume 2026, pages 117714–117737, 2026.

[29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[30] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.

[31] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via closed-loop resampling. *arXiv preprint arXiv:2408.17355*, 2024.

[32] Yufeng Liu, Hang Yu, Juntu Zhao, Bocheng Li, Di Zhang, Mingzhu Li, Wenxuan Wu, Yingdong Hu, Junyuan Xie, Junliang Guo, et al. Learning native continuation for action chunking flow policies. *arXiv preprint arXiv:2602.12978*, 2026.

[33] Jieting Long, Dechuan Liu, Weidong Cai, Ian Manchester, and Weiming Zhi. Constraining streaming flow models for adapting learned robot trajectory distributions. *arXiv preprint arXiv:2602.15567*, 2026.

[34] Rhea Malhotra, Yuejiang Liu, and Chelsea Finn. Self-guided action diffusion. *arXiv preprint arXiv:2508.12189*, 2025.

[35] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.

[36] Xiuxiu Qi, Yu Yang, Jiannong Cao, Luyao Bai, Chongshan Fan, Chengtai Cao, and Hongpeng Wang. Continuous vision-language-action co-learning with semantic-physical alignment for behavioral cloning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 24900–24908, 2026.

[37] Delin Qu, Haoming Song, Qizhi Chen, Zhaoqing Chen, Xianqiang Gao, Xinyi Ye, Qi Lv, Modi Shi, Guanghui Ren, Cheng Ruan, et al. Eo-1: Interleaved vision-text-action pretraining for general robot control. *arXiv preprint arXiv:2508.21112*, 2025.

[38] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.

[39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[40] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Jun Ma, and Haoang Li. Accelerating vision-language-action model integrated with action chunking via parallel decoding. *arXiv preprint arXiv:2503.02310*, 2025.

[41] Wenxuan Song, Ziyang Zhou, Han Zhao, Jiayi Chen, Pengxiang Ding, Haodong Yan, Yuxin Huang, Feilong Tang, Donglin Wang, and Haoang Li. Reconvla: Reconstructive vision-language-action model as effective robot perceiver. *arXiv preprint arXiv:2508.10333*, 2025.

[42] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.

[43] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[44] Haoxuan Wang, Gengyu Zhang, Yan Yan, Yuzhang Shang, Ramana Kompella, and Gaowen Liu. Real-time robot execution with masked action chunking. In *International Conference on Learning Representations*, volume 2026, pages 120161–120179, 2026.

[45] Yating Wang, Haoyi Zhu, Mingyu Liu, Jiange Yang, Hao-Shu Fang, and Tong He. Vq-vla: Improving vision-language-action models via scaling vector-quantized action tokenizers. *ArXiv*, abs/2507.01016, 2025.

[46] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.

[47] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026.

[48] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026.

[49] Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, et al. Qwen-robotmanip technical report: Alignment unlocks scale for robotic manipulation foundation models. *arXiv preprint arXiv:2606.17846*, 2026.

[50] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1702–1713, 2025.

[51] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[52] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 60580–60606, 2026.

[53] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

# Supplementary Material

## A Additional Implementation Details of ODEAction

This section supplements the implementation details in Sec. 2 of the main paper, describing the action head's inputs and outputs, the construction of demonstration supervision, the discretization of evolution time, integration and execution, and global parameters. ODEAction's continuous evolution, target field, and loss are defined in Eqs. (1)–(3) of the main paper, respectively, and the Euler recursion is given in Eq. (4) of the main paper.

**Action Head Inputs and Outputs.** Given visual observations $o$, a language instruction $l$, and robot proprioceptive information $s$, the vision-language backbone produces a conditioning representation $c = \phi(o, l, s)$. The ODEAction action head takes the current action $a$, evolution time $\tau$, and condition $c$ as inputs and outputs a transition vector $F_\theta(a, \tau \mid c) \in \mathbb{R}^d$ with the same dimensionality as the action. This vector represents the rate of change of the action with respect to evolution time, rather than the next action itself or a fixed-length action chunk. When used with the base policies in Sec. 3 of the main paper, the action representation, visual conditioning, and low-level control interface follow the configuration of the corresponding matched comparison group.

**Evolution Time, Interpolation, and Neighborhood Supervision.** Using control steps as discrete time units, let $t_o$ denote the initial control step aligned with the current conditioning representation $c$. An action evolution window spans $T_w$ control steps, giving

$$\tau = \frac{t - t_o}{T_w}, \qquad \Delta\tau = \frac{1}{T_w}. \tag{S1}$$

Training demonstrations consist of discrete action sequences. We perform linear interpolation between adjacent demonstrated actions along $\tau$ to obtain $\gamma(\tau)$. Within each interpolation segment, $\dot{\gamma}(\tau) = \mathrm{d}\gamma(\tau)/\mathrm{d}\tau$ is the corresponding piecewise evolution rate; derivatives at discrete demonstration nodes are assigned according to the convention for adjacent interpolation segments. During training, we sample $\tau \sim \mathcal{U}[0, 1]$ and $\epsilon \sim \mathcal{N}(0, I)$, and set

$$a = \gamma(\tau) + \sigma\epsilon, \qquad F^*(a, \tau) = \dot{\gamma}(\tau) + k\big(\gamma(\tau) - a\big). \tag{S2}$$

Neighborhood perturbations are used only to construct supervised query positions, rather than as starting points for generative denoising. Optimization uses the direct transition-field supervision in Eq. (3) of the main paper, and training itself does not require backpropagation through the numerical ODE solution process.

**Euler Integration, Observation Updates, and Action Anchors.** During execution, we query the action head at the current action $a_n$ and use the forward Euler method to compute

$$a_{n+1} = a_n + \Delta\tau F_\theta(a_n, \tau_n \mid c). \tag{S3}$$

Each integration step corresponds to one control step, and the generated $a_{n+1}$ becomes the action starting point for the next query.

When a new visual observation becomes available, the backbone updates the conditioning representation $c$ and continues integrating from the current action along the action transition field under the updated condition. Action integration and visual-observation updates follow the control frequency and observation schedule, respectively.

**Global Parameter Configuration.** ODEAction's neighborhood scale and correction gain control the extent of supervision coverage around demonstrations and the strength of recovery for deviating

actions, respectively. We perform a grid search over candidate configurations with $k \in [1, 10]$ and $\sigma \in [0.01, 1.0]$, and select $\sigma = 0.1$ and $k = 5$. The full method uses fixed $\sigma$ and $k$ throughout the entire evolution interval. Except for dedicated ablation experiments with $\sigma = 0$ or $k = 0$, this parameter configuration is used across all simulation benchmarks, base policies, and real-robot experiments in this paper.

## B Experimental Configurations and Evaluation Protocols

**Representative RoboTwin Tasks.** Controlled comparisons on RoboTwin [12] use five representative tasks: Adjust Bottle, Handover Block, Open Laptop, Lift Pot, and Place Can Basket. These cover pose-constrained fine adjustment with a single arm, bimanual handover, articulated-object manipulation, synchronized bimanual transport, and alternating bimanual manipulation, respectively. The five tasks span different motion constraints, bimanual coordination patterns, and end-effector manipulation precision requirements, and are used for the controlled comparisons and analyses in Secs. 3.4 and 3.5 of the main paper and Supplementary Materials F and G.

**Action Generation and Execution Configurations.** To compare different action generation methods under the same execution conditions, we keep the vision-language backbone, task-training data, action representation, control frequency, and visual-observation update times identical within each matched comparison. For action-chunking policies, we distinguish the length $H$ of an action chunk predicted in a single pass, the number of control steps actually executed $K_{\text{exec}}$, and the visual-observation update interval $K_{\text{obs}}$. RTC [9] and Legato [32] retain the action-chunk generation and execution interfaces of their corresponding base policies, introducing cross-chunk continuation mechanisms during inference and training, respectively. In ODEAction, the action head outputs a single-step action evolution rate, and control commands are generated through forward integration. All compared methods execute on matched control timelines and update visual observations at the same control times.

**Real-Robot Platform and Demonstration Data.** Experiments use an AgileX Cobot Magic 1.0 dual-arm robot equipped with Intel RealSense D435 and Orbbec DaBai RGB-D cameras. Model training and inference use an NVIDIA H100 GPU. Through teleoperation, we collect 100 demonstration trajectories for each of four tasks—Put Bananas, Open Drawer, Place Cup, and Insert Spoon—for a total of 400 trajectories. Each demonstration contains visual observations, robot proprioceptive information, and control actions aligned with a language instruction. The four compared methods are trained on the same task data.

**Real-Robot Evaluation Protocol.** Using $\pi_{0.5}$ [8] as the common base policy, we evaluate Base Policy, Base Policy + RTC, Base Policy + Legato, and Base Policy + ODEAction. Each method is tested 20 times on each task, yielding a total of 320 real-robot evaluations across the four methods and four tasks. All tasks use binary success criteria: Put Bananas requires correctly placing the bananas on the plate; Open Drawer requires fully opening the designated drawer; Place Cup requires correctly placing a water-filled cup on the plate without spilling water throughout execution; and Insert Spoon requires precisely inserting the spoon into the cup. Task success rates are calculated across all evaluation episodes, while the four control-command trajectory-quality metrics are computed only from the successful episodes of each method; detailed definitions are provided in Supplementary Material C. Representative executions of the four tasks are shown in Fig. S1.

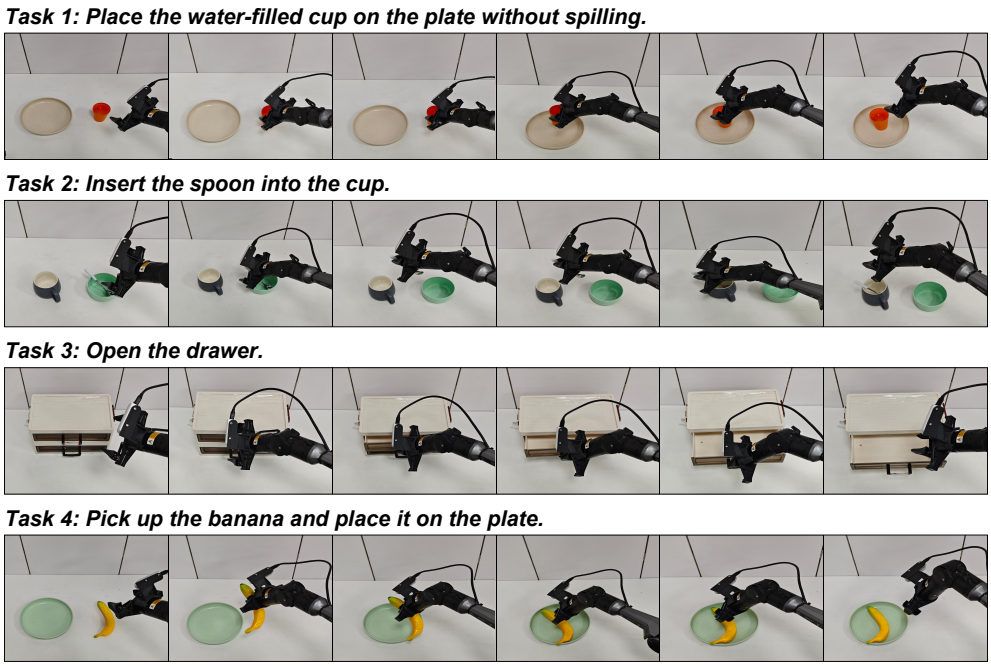



**Figure S1. Representative execution sequences for real-robot tasks.** Each row corresponds to one task, with frames showing key stages of execution from left to right.

## C Evaluation Metrics

**Measurement Targets and Sample Scope.** Success rates are calculated across all evaluation episodes. NLDLJ, NSPARC, EE-MAJ, and Boundary Jump are computed only for **the successful episodes of each method**, using the control-command sequences directly output by the action head, rather than high-frequency interpolations from the low-level controller or measured execution trajectories after its filtering. Success rates and the four trajectory metrics characterize task completion capability and command behavior during successful execution, respectively. Offline rollout errors in Sec. 3.5 of the main paper and Supplementary Material G use common demonstration-trajectory initial conditions, fixed conditioning, and evolution-time grids, and are evaluated independently of online evaluation episodes. Cross-task results are averaged over the tasks listed in each table.

NLDLJ, NSPARC, and EE-MAJ measure the **Motion Smoothness** of action trajectories. **Boundary Jump** measures **Trajectory Continuity** at the action-chunk boundary times of Base Policy; offline rollout error and the ratio of second-half to first-half error characterize the degree of error accumulation during recursive generation.

**Negative Log Dimensionless Jerk (NLDLJ).** NLDLJ characterizes the higher-order smoothness of an entire action trajectory in the time domain [5]. Jerk is the third temporal derivative of the action trajectory on the control-step timeline, i.e., the rate of change of acceleration. For an action trajectory of duration $T$, let $v(t)$ denote its scalar speed, $j(t)$ its jerk, and $v_{\text{peak}} = \max_t |v(t)|$ its peak speed. NLDLJ is defined as

$$\text{NLDLJ} = \log\left(\frac{T^5}{v_{\text{peak}}^2}\int_0^T \|j(t)\|_2^2\, dt\right). \tag{S4}$$

A lower NLDLJ indicates weaker higher-order changes and smoother overall motion. In practice, we compute NLDLJ on joint-action sequences. Gripper actions are excluded because they typically contain discrete opening and closing transitions and do not directly reflect the smoothness of the robotic arm's motion trajectory.

**Negative Spectral Arc Length (NSPARC).** NSPARC characterizes trajectory smoothness in the frequency domain and is sensitive to oscillations, repeated corrections, and high-frequency velocity changes [5]. We first obtain a scalar speed sequence from the Euclidean norms of difference vectors between adjacent actions in the joint-action sequence, and then compute the negative spectral arc length of this speed sequence. Gripper dimensions are also excluded. Given a speed signal $v(t)$ with spectral magnitude $|V(\omega)|$, the normalized spectrum is defined as

$$\widehat{V}(\omega) = \frac{|V(\omega)|}{|V(0)|}. \tag{S5}$$

The cutoff frequency is adaptively defined as the highest frequency at which the normalized spectral magnitude remains above 0.05. Denote this cutoff frequency by $\omega_c$, and let $\tilde{\omega} = \omega/\omega_c$. NSPARC is defined as

$$\mathrm{NSPARC}(v) = \int_0^{\omega_c} \left[ \left( \frac{\mathrm{d}\tilde{\omega}}{\mathrm{d}\omega} \right)^2 + \left( \frac{\mathrm{d}\widehat{V}(\omega)}{\mathrm{d}\omega} \right)^2 \right]^{1/2} \mathrm{d}\omega. \tag{S6}$$

A lower NSPARC indicates that velocity changes are more concentrated in the low-frequency region, with fewer oscillations and high-frequency corrections.

**End-Effector Mean Absolute Jerk (EE-MAJ).** EE-MAJ directly measures the mean absolute jerk of the end-effector trajectory, characterizing the absolute magnitude of abrupt action changes over the entire trajectory. Let $x(t)$ denote the end-effector position trajectory. Then

$$\text{EE-MAJ} = \frac{1}{T} \int_0^T \left\| \frac{\mathrm{d}^3 x(t)}{\mathrm{d}t^3} \right\|_2 \mathrm{d}t. \tag{S7}$$

For systems with multiple end effectors, we compute EE-MAJ separately for each end effector and then average the values. A lower EE-MAJ indicates fewer abrupt changes in the end-effector trajectory.

**Boundary Jump.** Boundary Jump measures the discontinuity of control commands when an action-chunking policy switches between adjacent action chunks. Let $B_{\mathrm{Base}}$ denote the set of action-chunk boundary times of Base Policy, and $a_t$ the action output at time $t$. Then

$$J^{\mathrm{bnd}} = \frac{1}{|B_{\mathrm{Base}}|} \sum_{t \in B_{\mathrm{Base}}} \|a_t - a_{t-1}\|_2. \tag{S8}$$

A lower value indicates more continuous action transitions at chunk boundaries. Because ODEAction does not contain action chunks, its Boundary Jump is computed at the same action-chunk boundary times as the corresponding Base Policy, ensuring that the two methods are compared at the same execution times. RTC, Legato, and direct action-by-action prediction also use the boundary times of the original Base Policy in the matched comparison as common reference times; these boundary times are determined by the actual execution schedule of the corresponding Base Policy.

**Offline Rollout Error and Error Accumulation Ratio.** Offline rollout error characterizes the deviation of the learned action transition field from demonstration trajectories during recursive generation. Given a demonstration trajectory $\gamma$, we initialize the rollout at its starting point, recursively generate actions using ODEAction's action transition field under fixed conditioning, and compare each generated action with the demonstrated action at the same evolution time. The full-rollout error is defined as

$$E = \mathbb{E}\left[ \frac{1}{Md} \sum_{n=1}^{M} \|\hat{a}_n - \gamma(n\Delta\tau)\|_2^2 \right], \tag{S9}$$

where $\hat{a}_n$ is the $n$-th recursively generated action, $M$ is the number of rollout steps, and $d$ is the action dimensionality, consistent with the definition in Supplementary Material A; the expectation is taken over the demonstration trajectories used for offline evaluation. We further compute the same type of error over the first and second halves of evolution time, denoted by $E_1$ and $E_2$, respectively, and use $E_2/E_1$ as the **Error Accumulation Ratio**. A ratio greater than 1 indicates that errors are further amplified in the second half of the rollout; a ratio close to or below 1 indicates that recursive error accumulation is effectively suppressed.

## D Statistical Association Between BDR and Success-Rate Gains

To quantitatively examine the relationship between **Boundary Discontinuity Reduction (BDR)** and task-performance gains in Sec. 3.2 of the main paper, we use the four LIBERO suites and six EBench tasks as a total of 10 evaluation units. BDR is computed according to Eq. (5) of the main paper. For the $i$-th evaluation unit, the success-rate gain, expressed in percentage points, is defined as

$$\Delta \mathrm{SR}_i = \mathrm{SR}_i^{\mathrm{ODEAction}} - \mathrm{SR}_i^{\mathrm{Base}}. \tag{S10}$$

Based on the evaluation-unit results reported in Table 1 of the main paper, we perform a two-sided Spearman rank correlation test between BDR and success-rate gains, obtaining

$$\rho_s = 0.939, \qquad p = 5.61 \times 10^{-5}. \tag{S11}$$

The results show a statistically significant strong positive correlation between BDR and success-rate gains: evaluation units with greater reductions in boundary discontinuity generally also achieve larger task-performance gains. This statistical result supports the observation in Sec. 3.2 of the main paper that improvements in trajectory continuity and gains in task performance exhibit a consistent positive association across tasks.

## E Per-Task Results on RoboTwin and Real Robots

To supplement the aggregate RoboTwin results in Table 2 of the main paper, Table S1 reports the per-task success rates of **Base Policy + ODEAction** on all 50 tasks under the Clean and Random evaluation settings. The average success rates are **89.44%** and **89.84%**, respectively, corresponding to the rounded values of **89.4%** and **89.8%** in the main paper.

**Real-Robot Per-Task Results.** Figure 4(c–d) of the main paper shows the average success rate and trajectory quality of each method across the four real-robot tasks. Table S2 further provides the complete quantitative results for the four methods on each task, including success rate, NLDLJ, NSPARC, EE-MAJ, and Boundary Jump. The experimental platform, success criteria, and evaluation protocol are described in Supplementary Material B, and the calculation of trajectory-quality metrics is provided in Supplementary Material C.

**Table S1. Per-task success rates (%) of Base Policy + ODEAction on all 50 RoboTwin tasks.** C and R denote the Clean and Random evaluation settings, respectively.

| Task | C | R | Task | C | R |
|---|---|---|---|---|---|
| Adjust Bottle | 100 | 100 | Place Can Basket | 74 | 74 |
| Beat Block Hammer | 95 | 94 | Place Cans Plastic Box | 100 | 99 |
| Blocks Ranking RGB | 99 | 98 | Place Container Plate | 99 | 99 |
| Blocks Ranking Size | 85 | 86 | Place Dual Shoes | 89 | 88 |
| Click Alarm Clock | 73 | 77 | Place Empty Cup | 97 | 97 |
| Click Bell | 84 | 87 | Place Fan | 91 | 92 |
| Dump Bin Big Bin | 100 | 100 | Place Mouse Pad | 73 | 79 |
| Grab Roller | 100 | 100 | Place Object Basket | 87 | 88 |
| Handover Block | 94 | 91 | Place Object Scale | 89 | 89 |
| Handover Mic | 100 | 99 | Place Object Stand | 99 | 98 |
| Hanging Mug | 55 | 52 | Place Phone Stand | 71 | 75 |
| Lift Pot | 100 | 100 | Place Shoe | 96 | 99 |
| Move Can Pot | 80 | 79 | Press Stapler | 100 | 98 |
| Move Pill Bottle Pad | 98 | 99 | Put Bottles Dustbin | 87 | 83 |
| Move Playing Card Away | 98 | 97 | Put Object Cabinet | 93 | 94 |
| Move Stapler Pad | 56 | 70 | Rotate QR Code | 88 | 89 |
| Open Laptop | 99 | 100 | Scan Object | 92 | 90 |
| Open Microwave | 97 | 100 | Shake Bottle | 100 | 100 |
| Pick Diverse Bottles | 85 | 84 | Shake Bottle Horizontally | 100 | 100 |
| Pick Dual Bottles | 88 | 89 | Stack Blocks Three | 96 | 91 |
| Place A2B Left | 90 | 94 | Stack Blocks Two | 100 | 100 |
| Place A2B Right | 87 | 92 | Stack Bowls Three | 82 | 79 |
| Place Bread Basket | 89 | 87 | Stack Bowls Two | 99 | 100 |
| Place Bread Skillet | 95 | 90 | Stamp Seal | 67 | 71 |
| Place Burger Fries | 99 | 99 | Turn Switch | 57 | 56 |
| **Average** | | | | **89.44** | **89.84** |

**Table S2. Comparison of success rates and control-command trajectory quality for four methods on four real-robot tasks.** With $\pi_{0.5}$ as the base policy, values in each cell correspond to **Base Policy / +RTC / +Legato / +ODEAction**, in that order. Higher SR is better, while lower values are better for the other four metrics; EE-MAJ is reported in units of $10^{-3}$, and bold indicates the best results. Success rates are calculated from 20 trials per method per task, while trajectory-quality metrics are computed only from the successful episodes of each method.

| Task | SR (%) ↑ | NLDLJ ↓ | NSPARC ↓ | EE-MAJ ↓ | Boundary Jump ↓ |
|---|---|---|---|---|---|
| Put Bananas | 70.00 / 60.00 / 70.00 / **75.00** | 25.84 / 25.38 / 25.17 / **24.66** | 2.757 / 2.691 / 2.656 / **2.595** | 2.62 / 2.49 / 2.42 / **1.79** | 0.1562 / 0.1519 / 0.1532 / **0.1474** |
| Open Drawer | 95.00 / 95.00 / 90.00 / **100.00** | 24.98 / 24.17 / 24.34 / **22.47** | 2.873 / 2.814 / 2.829 / **2.734** | 3.24 / 2.87 / 2.95 / **2.33** | 0.1147 / 0.1108 / 0.1089 / **0.1027** |
| Place Cup | 70.00 / 65.00 / 65.00 / **90.00** | 24.86 / 24.25 / 23.99 / **23.01** | 2.365 / 2.341 / 2.318 / **2.296** | 3.06 / 2.77 / 2.56 / **1.42** | 0.1517 / 0.1434 / 0.1411 / **0.1374** |
| Insert Spoon | 25.00 / 35.00 / 40.00 / **55.00** | 25.03 / 24.74 / 24.77 / **23.92** | 2.146 / 2.125 / 2.130 / **2.097** | 3.13 / 2.74 / 2.69 / **1.68** | 0.1455 / 0.1406 / 0.1419 / **0.1365** |
| **Average** | 65.00 / 63.75 / 66.25 / **80.00** | 25.18 / 24.64 / 24.57 / **23.52** | 2.535 / 2.493 / 2.483 / **2.431** | 3.01 / 2.72 / 2.66 / **1.81** | 0.1420 / 0.1367 / 0.1363 / **0.1310** |

## F Direct Next-Action Prediction and Continuous Action Evolution

ODEAction transforms action generation from future action-chunk prediction into continuous evolution starting from the current action. However, simply removing action chunks and establishing recursive dependencies between adjacent actions does not imply that stable action evolution dynamics have been learned. To examine whether ODEAction's benefits arise solely from this chunk-free recursive formulation, we compare it with Direct Next-Action prediction (Direct Next-Action). Both methods generate subsequent actions step by step, but use different learning objectives and generation mechanisms.

**Direct Next-Action Prediction (Direct Next-Action).** Given the current action, evolution time, and vision-language conditioning, Direct Next-Action directly regresses the next action value, with the recursion

$$\hat{a}_{n+1} = G_\psi(\hat{a}_n, \tau_n \mid c), \tag{S12}$$

where $\psi$ denotes the parameters of the prediction network $G_\psi$. Supervised by adjacent action pairs from demonstrations, this network learns a direct mapping from the current action to the next action and uses the predicted action as the input to the next step during inference. In contrast, ODEAction learns an action evolution rate jointly defined by the demonstration tangent and the deviation correction term, and advances actions through ODE integration. Its supervision also covers demonstration neighborhoods, providing corresponding evolution directions for off-trajectory actions.

We use the StarVLA-$\pi$ base policy and five representative RoboTwin tasks from Sec. 3.4 of the main paper, keeping the vision-language backbone, training data, control frequency, and visual-observation update schedule identical. Task performance is measured by success rate, and control-command trajectory quality is measured by NLDLJ, NSPARC, EE-MAJ, and Boundary Jump, following the same statistical protocol as Sec. 3.1 of the main paper.

**Table S3. Task performance and control-command trajectory quality of Direct Next-Action and ODEAction on five representative RoboTwin tasks.** Values in each cell are ordered as **Direct Next-Action / ODEAction**, and bold indicates the best results. Higher SR is better, while lower values are better for the remaining metrics; EE-MAJ is reported in units of $10^{-3}$.

| Task | SR (%) ↑ | NLDLJ ↓ | NSPARC ↓ | EE-MAJ ↓ | Boundary Jump ↓ |
|---|---|---|---|---|---|
| Adjust Bottle | 92.0 / **94.0** | 32.87 / **21.54** | 3.891 / **1.991** | 5.614 / **0.625** | 0.2914 / **0.1507** |
| Handover Block | 67.0 / **82.0** | 37.26 / **26.67** | 4.622 / **2.735** | 8.327 / **0.581** | 0.2731 / **0.0391** |
| Open Laptop | 83.0 / **87.0** | 34.95 / **22.73** | 4.107 / **1.848** | 6.923 / **0.449** | 0.3236 / **0.0264** |
| Lift Pot | 88.0 / **91.0** | 33.58 / **20.69** | 3.673 / **1.566** | 8.541 / **0.511** | 0.4338 / **0.1613** |
| Place Can Basket | 62.0 / **68.0** | 35.76 / **23.18** | 4.414 / **2.338** | 6.350 / **0.500** | 0.2995 / **0.0462** |
| **Average** | 78.4 / **84.4** | 34.88 / **22.96** | 4.141 / **2.096** | 7.151 / **0.533** | 0.3243 / **0.0847** |

**Results.** As shown in Table S3, Direct Next-Action achieves an average success rate of 78.4%, indicating that direct action-by-action prediction can complete a substantial portion of manipulation tasks, but the trajectory quality of its generated actions is markedly worse than that of ODEAction. ODEAction raises the average success rate to 84.4%, an improvement of 6.0 percentage points, while improving all four trajectory metrics on all five tasks: average NLDLJ and NSPARC decrease from 34.88 and 4.141 to 22.96 and 2.096, respectively; EE-MAJ decreases from 7.151 to 0.533; and Boundary Jump decreases from 0.3243 to 0.0847. These results indicate that, although direct next-action prediction establishes generative dependencies between adjacent actions, its recursive process remains prone to pronounced command jitter and abrupt action changes.

This comparison further clarifies ODEAction's modeling advantage: **chunk-free action-by-action recursion alone is insufficient to achieve high-quality continuous action generation.** Direct Next-Action

directly learns a mapping between adjacent demonstrated actions, whereas ODEAction formulates action generation as a continuous evolution process, characterizes evolution rates by learning an action transition field, and combines neighborhood supervision with deviation correction to provide evolution directions after deviations from demonstrations. Together with the component ablation results in Sec. 3.5 of the main paper, these findings show that ODEAction's gains in task performance and trajectory quality arise not only from removing action chunks, but are also closely related to continuous action evolution modeling and its corresponding field-supervision design.

## G Neighborhood Supervision and Recursive Behavior of the Action Transition Field

ODEAction models action generation as continuous evolution starting from the current action, using an action transition field to jointly characterize motion progression along demonstration trajectories and directional correction after deviations. During training, we sample demonstration neighborhoods at the same scale across different evolution times, allowing the action transition field to learn evolution directions for deviations from demonstrations throughout the entire evolution interval. Given a demonstration action curve $\gamma(\tau)$, ODEAction's training query distribution is

$$q_{\mathrm{U}}(a \mid \tau, \gamma) = \mathcal{N}\big(\gamma(\tau), \sigma^2 I\big). \tag{S13}$$

The corresponding target action transition field is

$$F^{*}_{\mathrm{U}}(a, \tau) = \dot{\gamma}(\tau) + k\big(\gamma(\tau) - a\big). \tag{S14}$$

Here, the demonstration tangent $\dot{\gamma}(\tau)$ provides the direction of action progression, while the correction term $k(\gamma(\tau) - a)$ provides a recovery direction for query actions that deviate from the demonstration. Time-Uniformity here means that the neighborhood scale of the training query distribution does not vary with evolution time, rather than requiring the variance of the action distribution to remain constant during integration. Training can thus continuously cover demonstration neighborhoods of the same scale at different stages of evolution.

Jointly incorporating motion progression and recovery from deviations into dynamical systems has a long history in robot control. Dynamical Movement Primitives (DMP) generate goal-directed motion through attractor dynamical systems and learnable motion-shaping terms [17]; Stable Estimator of Dynamical Systems (SEDS) learns nonlinear motion fields that converge to a goal under stability constraints [21]. These studies embody the basic idea of using dynamical systems to organize motion generation and recovery from deviations. ODEAction applies this idea to action transition field learning conditioned on visual, language, and proprioceptive information. Through demonstration-induced local supervision, action generation can proceed recursively along the learned evolution rates.

Streaming Flow Policy (SFP) [19] also constructs a stabilized velocity field using the demonstration tangent and proportional deviation correction, in the form

$$v_{\gamma}(a, \tau) = \dot{\gamma}(\tau) + k\big(\gamma(\tau) - a\big). \tag{S15}$$

Starting from a Gaussian distribution near the demonstration's initial point, SFP obtains a conditional action distribution that gradually contracts over evolution time under the stabilized field above [19]:

$$q_{\mathrm{S}}(a \mid \tau, \gamma) = \mathcal{N}\big(\gamma(\tau), \sigma_0^2 e^{-2k\tau} I\big)\,, \qquad \sigma_{\mathrm{S}}(\tau) = \sigma_0 e^{-k\tau}. \tag{S16}$$

Here, $\sigma_0$ is the initial neighborhood standard deviation, and $\sigma_{\mathrm{S}}(\tau)$ is the neighborhood standard deviation at evolution time $\tau$. SFP uses this conditional distribution for flow matching, focusing on

aligning the flow sampling process with action execution time so that actions can be executed directly in a streaming manner during generation, thereby reducing the overhead of waiting for a complete action sequence to be generated. This method has primarily been studied in visuomotor imitation learning settings such as Push-T, RoboMimic, and single-arm robots [19]. ODEAction, in contrast, addresses action-chunk generation in vision-language-action models, replacing joint prediction of future action chunks with continuous action evolution and examining its effects on task performance, action-trajectory quality, and recursive error. The two methods share the same local corrective field form, but differ in their research questions, base policies, and the temporal distribution of training neighborhoods.

To examine the effect of the temporal distribution of neighborhoods on action transition field learning, we build a Shrinking Construction with SFP-style exponential contraction and conduct a controlled comparison with the Uniform Construction used by ODEAction. Both constructions use the same target action transition field and the StarVLA-$\pi$ base policy and five representative RoboTwin tasks from Sec. 3.4 of the main paper, differing in how the neighborhood scale of the training query distribution varies with evolution time. Below, we refer to Uniform Construction as Uniform ODEAction and compare task success rates, control-command trajectory quality, and offline recursive error.

**Table S4. Task performance and control-command trajectory quality of Shrinking Construction and Uniform ODEAction on five representative RoboTwin tasks.** Values in each cell are ordered as **Shrinking / Uniform ODEAction**, and bold indicates the best results. Higher SR is better, while lower values are better for the remaining metrics; EE-MAJ is reported in units of $10^{-3}$. The four trajectory-quality metrics are computed from the successful episodes of each method.

| Task | SR (%) ↑ | NLDLJ ↓ | NSPARC ↓ | EE-MAJ ↓ | Boundary Jump ↓ |
|---|---|---|---|---|---|
| Adjust Bottle | 91.0 / **94.0** | 22.31 / **21.54** | 2.004 / **1.991** | 0.813 / **0.625** | 0.1585 / **0.1507** |
| Handover Block | 74.0 / **82.0** | 28.36 / **26.67** | 2.865 / **2.735** | 1.061 / **0.581** | 0.0594 / **0.0391** |
| Open Laptop | 86.0 / **87.0** | 24.91 / **22.73** | 1.955 / **1.848** | 0.923 / **0.449** | 0.0695 / **0.0264** |
| Lift Pot | 89.0 / **91.0** | 21.78 / **20.69** | 1.638 / **1.566** | 0.781 / **0.511** | 0.1824 / **0.1613** |
| Place Can Basket | 53.0 / **68.0** | 24.71 / **23.18** | 2.494 / **2.338** | 0.750 / **0.500** | 0.0655 / **0.0462** |
| **Average** | 78.6 / **84.4** | 24.41 / **22.96** | 2.191 / **2.096** | 0.866 / **0.533** | 0.1071 / **0.0847** |

Table S4 shows that the time-uniform neighborhood construction achieves better overall performance across the five tasks: the average success rate increases from 78.6% for Shrinking to 84.4% for ODEAction, while NLDLJ, NSPARC, EE-MAJ, and Boundary Jump also improve on all five tasks. In particular, success rates on Place Can Basket and Handover Block increase by 15 and 8 percentage points, respectively; average EE-MAJ decreases from 0.866 to 0.533, and Boundary Jump from 0.1071 to 0.0847. Because both constructions use the same local target field, these differences indicate that the distribution of training supervision over evolution time itself affects the task performance and trajectory quality of action generation.

We further use the offline demonstration-trajectory protocol in Supplementary Material C to examine the recursive error behavior of the two constructions. Starting from the same demonstration initial conditions, the models recursively generate actions under fixed conditioning and compare them with the demonstrated actions at corresponding evolution times. Table S5 reports the full-trajectory error $E$, first-half error $E_1$, second-half error $E_2$, and their ratio $E_2/E_1$. Unlike online command-trajectory metrics, offline errors are computed on a common evaluation set of demonstration trajectories.

Table S5 further reveals differences between the two supervision constructions during recursion. The average full-trajectory error is 7.52 for Shrinking and 4.25 for Uniform ODEAction; their first-half errors are 5.87 and 4.42, respectively, with the gap widening further in the second half to 9.17 and 4.07. Shrinking has an error accumulation ratio of 1.56, with second-half error growth on all five tasks; Uniform ODEAction has an error accumulation ratio of 0.92, with second-half errors lower than

**Table S5. Offline recursive errors of Shrinking Construction and Uniform ODEAction on five representative RoboTwin tasks.** Values in each cell are ordered as **Shrinking / Uniform ODEAction**, and bold indicates the best results. $E$, $E_1$, and $E_2$ are reported in units of $10^{-4}$, with lower values being better for all three; a lower $E_2/E_1$ indicates weaker relative error growth in the second half. The ratio in the Average row is obtained by dividing the cross-task average of $E_2$ by the cross-task average of $E_1$.

| Task | $E \downarrow$ | $E_1 \downarrow$ | $E_2 \downarrow$ | $E_2/E_1 \downarrow$ |
|---|---|---|---|---|
| Adjust Bottle | 5.55 / **2.90** | 4.40 / **3.14** | 6.69 / **2.65** | 1.52 / **0.84** |
| Handover Block | 8.64 / **4.68** | 7.05 / **5.03** | 10.22 / **4.32** | 1.45 / **0.86** |
| Open Laptop | 7.39 / **3.79** | 5.35 / **3.90** | 9.42 / **3.67** | 1.76 / **0.94** |
| Lift Pot | 5.66 / **3.15** | 4.60 / **3.38** | 6.72 / **2.91** | 1.46 / **0.86** |
| Place Can Basket | 10.38 / **6.73** | 7.95 / **6.65** | 12.80 / **6.80** | 1.61 / **1.02** |
| **Average** | 7.52 / **4.25** | 5.87 / **4.42** | 9.17 / **4.07** | 1.56 / **0.92** |

first-half errors on all tasks except Place Can Basket, whose ratio is close to 1. Together with the results in Table S4, time-uniform neighborhood supervision not only improves task completion capability and control-command trajectory quality, but also exhibits better control of recursive error in the evaluated settings. This is consistent with the design of using a fixed neighborhood scale to continuously provide supervision for off-trajectory actions throughout the entire evolution interval, further supporting ODEAction's choice of action transition field construction.